

Week 13 – 2D Arrays & Grids

■ Learning Objectives

- Understand 2D list (matrix) representation and access.
- Practice nested loops for grid traversal.
- Apply algorithms to count, sum, and simulate movement in grids.
- Implement basic pathfinding using recursion or BFS.

Example 1 – Print 2D Grid

```
grid = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
for r in range(len(grid)):  
    for c in range(len(grid[r])):  
        print(grid[r][c], end=" ")  
    print()
```

Example 2 – Counting Cells

```
grid = [  
    ['#', '.', '.'],  
    ['.', '#', '#'],  
    ['#', '.', '.']  
]  
  
count = 0  
for r in range(len(grid)):  
    for c in range(len(grid[r])):  
        if grid[r][c] == '#':  
            count += 1  
print("Number of walls:", count)
```

Example 3 – Neighbour Counting

```
grid = [  
    ['#', '.', '#'],  
    ['.', '#', '.'],  
    ['#', '.', '.']  
]  
  
rows, cols = len(grid), len(grid[0])  
dirs = [(-1,0),(1,0),(0,-1),(0,1)]  
  
for r in range(rows):  
    for c in range(cols):  
        count = 0  
        for dr, dc in dirs:  
            nr, nc = r+dr, c+dc  
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == '#':  
                count += 1  
        print(count, end=" ")  
    print()
```

■ Pathfinding Challenge – Find Path in a Maze

```
maze = [
    ['S', '.', '#', '.'],
    ['.', '.', '#', 'G'],
    ['#', '.', '.', '.']
]

rows, cols = len(maze), len(maze[0])
dirs = [(-1,0), (1,0), (0,-1), (0,1)]

def dfs(r, c, visited):
    if r < 0 or r >= rows or c < 0 or c >= cols:
        return False
    if maze[r][c] == '#' or (r, c) in visited:
        return False
    if maze[r][c] == 'G':
        return True
    visited.add((r, c))
    for dr, dc in dirs:
        if dfs(r+dr, c+dc, visited):
            return True
    return False

print("Path found:", dfs(0, 0, set()))
```

■ Practice Problems

Practice 1 – Diagonal Sums

```
grid = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
main = anti = 0  
n = len(grid)  
for i in range(n):  
    main += grid[i][i]  
    anti += grid[i][n-i-1]  
print("Main:", main, "Anti:", anti)
```

Practice 2 – Transpose a Matrix

```
grid = [  
    [1, 2, 3],  
    [4, 5, 6]  
]  
  
rows, cols = len(grid), len(grid[0])  
transposed = [[grid[r][c] for r in range(rows)] for c in range(cols)]  
print(transposed)
```

■ Homework Challenge

Simulate a garden grid: each day, all cells increase by 1 except cells adjacent to a 0 (dead plant). Run for 3 days and print the final grid.

```
grid = [
    [2, 0, 3],
    [1, 2, 1],
    [0, 1, 2]
]

dirs = [(-1,0),(1,0),(0,-1),(0,1)]

for _ in range(3):
    new_grid = [row[:] for row in grid]
    for r in range(len(grid)):
        for c in range(len(grid[r])):
            if grid[r][c] == 0:
                continue
            for dr, dc in dirs:
                nr, nc = r+dr, c+dc
                if 0 <= nr < len(grid) and 0 <= nc < len(grid[r]) and grid[nr][nc] == 0:
                    break
            else:
                new_grid[r][c] += 1
    grid = new_grid

for row in grid:
    print(row)
```